

[GAME] THE LAST PRIMORDIAL — Complete Development Plan

A 2D action-adventure built in **Bevy (Rust)**, featuring seven unique realms, philosophical storytelling, pixel art visuals, and three branching endings.

Table of Contents

- Project Overview
- Development Phases
- Phase 0 — Pre-Production & Foundation
- Phase 1 — Core Engine & Vertical Slice
- Phase 2 — Prologue & Village (Chapter 1–2)
- Phase 3 — The Seven Realms (Chapter 3)
- Phase 4 — The Core & Endings (Chapters 4–5)
- Phase 5 — Polish, QA & Launch
- How to Add Animations
- How to Add Story Scenes
- How to Add Audio Tracks
- How to Add Story Clips (Cutscenes)
- How to Add Characters
- Dependency & Parallelism Map
- Tool & Asset Pipeline Summary

Project Overview

Field	Detail
Game Title	The Last Primordial
Genre	2D Action-Adventure / Narrative Puzzle
Engine	Bevy (Rust)
Art Style	Pixel Art + Hand-drawn elements
Audio	Orchestral + Pixel-style SFX + Realm-specific ambience
Platforms	PC (Steam) → Nintendo Switch → Mac / Linux
Price	\$19.99 – \$24.99 (premium buy-to-play)

Team	Surabhi Pandey, Jemin Morabiya, Tarun Ahuja
Protagonist	Kael (internally "Christ" in lore bible)
Key Mechanic	Seven Realms, each with unique gameplay rules
Endings	3 (Eternal Guardian / Age of Humanity / Broken Cycle)

Development Phases

Phase 0 — Pre-Production & Foundation (Weeks 1–6)

Goal: Establish project infrastructure, finalize design documents, and set up the Bevy engine scaffold.

[SEQ] Sequential Tasks (must be done in order)

#	Task	Details	Deliverable
0.1	Set up Rust + Bevy project	Initialize a Bevy 0.14+ project with <code>cargo init</code> . Configure <code>Cargo.toml</code> with bevy, bevy_ecs_idtk (for tilemap), bevy_kira_audio (audio), bevy_rapier2d (physics).	Compiling Bevy project with a blank window
0.2	Version Control	Initialize Git repo, set up GitHub with branch protection rules (<code>main</code> , <code>develop</code> , feature branches). Add <code>.gitignore</code> for Rust/Bevy.	GitHub repo with CI (cargo check, clippy)
0.3	Define project folder structure	Create standard directories: <code>src/</code> , <code>assets/sprites/</code> , <code>assets/tilesets/</code> , <code>assets/audio/</code> , <code>assets/dialogues/</code> , <code>assets/cutscenes/</code> , <code>assets/fonts/</code> .	Documented folder tree
0.4	Game Design Document (GDD) finalization	Merge DYNAMICS.pdf and GAME_LORE_BIBLE.pdf into a single living GDD. Add mechanic specs for each realm, UI wireframes, control mappings.	Living GDD (Markdown/Notion)

[PAR] Parallel Tasks (can happen simultaneously)

#	Task	Details	Deliverable
---	------	---------	-------------

0.5	Art Direction & Style Guide	Create pixel art style guide: palette per realm, character proportions (e.g. 32×48 px sprites), tile size (16×16 or 32×32), animation frame counts. Use reference images from Pinterest + hand-drawn sketches.	Style guide PDF + reference board
0.6	Audio Direction Document	Define music themes per realm, SFX categories (combat, UI, environment, boss), audio format (OGG Vorbis for music, WAV for SFX). Research orchestral sample libraries.	Audio design doc
0.7	Narrative Script	Convert lore bible into a dialogue script format: character ID, line, emotion tag, trigger condition. Use a simple JSON or RON format that Bevy can parse.	<code>dialogues/*.ron</code> template files
0.8	Tool Selection	Choose pixel art tool (Aseprite), tilemap editor (LDTk), audio DAW (LMMS/Reaper), cutscene tool (custom Bevy system or scripted).	Tool manifest document

[TIP] TIP: Tasks 0.5, 0.6, 0.7, and 0.8 are fully independent of each other and can be assigned to different team members working simultaneously. They only depend on 0.4 (GDD finalization) being complete.

Phase 1 — Core Engine & Vertical Slice (Weeks 7–18)

Goal: Build the core game systems and produce a playable vertical slice (one small area demonstrating combat, dialogue, movement, and a mini-boss).

[SEQ] Sequential Tasks

#	Task	Details	Deliverable
1.1	Player Movement System	Implement Kael's movement: walk, run, jump, dash, ledge grab. Use <code>bevy_rapier2d</code> for physics collisions. Sprite-based animation state machine (idle, walk, run, jump, fall, land).	Playable character on flat ground
1.2	Camera System	Smooth-follow camera with dead zone. Camera boundaries for rooms. Parallax scrolling background layers. Lerp-based camera transitions between rooms.	Camera follows player smoothly

1.3	Combat System (Basic)	Light attack, heavy attack, dodge roll, block/parry. Hitbox/hurtbox system using Rapier sensors. Damage calculation, knockback, i-frames.	Kael can attack and take damage
1.4	Enemy AI (Basic)	Patrol → detect → chase → attack state machine. Simple enemies: melee grunt, ranged archer. Pathfinding via raycasts (no navmesh needed for 2D).	2-3 enemy types with basic AI
1.5	Tilemap & Level Loading	Integrate LDtk for level design. Load <code>.ldtk</code> files via <code>bevy_ecs_ldtk</code> . Collision layer, entity layer, decoration layer.	One test level loading correctly

[PAR] Parallel Tasks

#	Task	Details	Deliverable
1.6	UI System	HUD: health bar, stamina bar, Black Sword dialogue indicator. Pause menu, settings menu. Inventory/pendant display. Use <code>bevy_ui</code> or <code>bevy_egui</code> .	Functional HUD overlay
1.7	Dialogue System	Parse <code>.ron</code> dialogue files. Display text box with character portrait, name, line. Support branching choices. Typewriter text effect. Voice within the Black Sword: multiple speaker portraits for past wielders.	Working dialogue box
1.8	Save/Load System	Serialize game state to JSON/RON: player position, realm progress, choice history, NPC states. Auto-save at realm transitions. Manual save at specific points.	Save/load working
1.9	Sprite Sheet Pipeline	Build asset loading system: load Aseprite-exported sprite sheets, define animation clips (frame indices, duration, loop mode). Atlas generation with <code>bevy_asset_loader</code> .	Automated sprite loading
1.10	Audio Engine Integration	Integrate <code>bevy_kira_audio</code> . Implement channels: music, SFX, ambient, dialogue. Volume control per channel. Crossfade between music tracks.	Audio playback working

[WARN] IMPORTANT: ****Vertical Slice Checkpoint (Week 18):**** By the end of this phase, you should have a playable demo area where Kael can move, fight enemies, talk to an NPC, hear music/SFX, and save/load the game. This is your proof-of-concept before investing in content.

Phase 2 — Prologue & Village (Chapters 1–2) (Weeks 19–28)

Goal: Build the game's opening: Kael's nightmare sequence, the village, NPC interactions, and the Black Sword discovery.

[SEQ] Sequential Tasks

#	Task	Details	Deliverable
2.1	Nightmare Sequence	Scripted cutscene: burning house, seven silhouettes, black sword, parents calling Kael's name. Use the cutscene system (see How to Add Story Clips). Screen shake, red tint, distorted audio.	Playable nightmare intro
2.2	Village Hub World	Design Kael's hometown in LDtk: houses, fields, village square, elder's home, training ground. NPCs with dialogue trees. Day/night cycle (cosmetic).	Explorable village
2.3	NPC Interaction System	Talk to villagers, receive information about parents. Different NPCs give different (conflicting) stories about how parents died. Relationship system (simple affinity scores).	NPCs with branching dialogue
2.4	Black Sword Discovery Event	Triggered event: sword appears outside Kael's home. Cinematic: hand touches hilt → world goes silent → voices of past wielders flood in → first memory flash. Player gains the Black Sword as weapon + narrative tool.	Pivotal story moment playable
2.5	Departure & Overworld Intro	Kael leaves village. Overworld map or interconnected zones leading toward the first Realm. Father's voice: "Hatred makes decisions easy. Truth makes them difficult."	Transition to main game

[PAR] Parallel Tasks

#	Task	Details	Deliverable
---	------	---------	-------------

2.6	Village Pixel Art Assets	All village tiles, house interiors, NPC sprites (idle, talk, walk), props (barrels, flowers, fences, market stalls).	Complete village tileset
2.7	Village Music & Ambience	Peaceful village theme (acoustic, warm), night variant, marketplace chatter SFX, nature ambience (birds, wind).	2-3 music tracks + ambient loops
2.8	Black Sword Voice System	Extend dialogue system: multiple "speakers" within the sword. Each past wielder has a unique portrait and text color. Occasionally they argue with each other. System tracks player choices to influence which wielders speak.	Sword voices working in dialogue
2.9	Nightmare VFX	Screen distortion shaders (chromatic aberration, vignette pulse, color desaturation). Fire particle effects. Silhouette rendering for the Seven.	VFX shaders ready

[TIP] TIP: Art tasks (2.6, 2.9) and audio tasks (2.7) can run in parallel with programming tasks (2.1–2.5). The Black Sword voice system (2.8) depends on the dialogue system (1.7) being complete.

Phase 3 — The Seven Realms (Chapter 3) (Weeks 29–52)

Goal: Build all seven realms. This is the largest phase. Each realm takes approximately **3–3.5 weeks** and follows the same production pipeline.

Per-Realm Production Pipeline (repeat 7 times)

Each realm follows a **sequential** internal pipeline, but **different realms can overlap** if you have the team capacity:

Week 1: Environment Design + Tileset Creation + Music Composition	Week 2: Unique Mechanics Implementation + Enemy Design	Week 3: Realm Spirit Boss Fight + Primordial Dialogue Scene	Week 0.5: Integration Testing + Bug Fixes
---	--	---	---

Realm Breakdown

Realm	Unique Mechanic	Boss (Realm Spirit)	Primordial	Key Dialogue
1. Time	Time acceleration, reversal, freeze, fracture zones	Spirit of Time (arena rewinds, ghostly echoes)	Chronar	"Which moment made you believe revenge could heal grief?"
2. Space	Impossible geometry, portals, perspective puzzles, gravity shifts	Spirit of Space (infinite geometric prison)	Vastara	"If you could place every person exactly where they belong... would anyone still be free?"

3. Life	Living environment, symbiotic creatures, ecosystem puzzles, regenerating enemies	Spirit of Life (refuses to let anything die)	Elaria	"Would you save every life... even if saving one meant condemning thousands?"
4. Death	Emotional encounters, helping souls, minimal combat, memory puzzles	Spirit of Death (drains strength, patience-based fight)	Morvex	"Do you hate me... because I brought death to your family... or because death reached them at all?"
5. Chaos	Randomized patterns, shifting controls, evolving puzzles, NPCs from wrong timelines	Spirit of Chaos (rules are prisons)	Xael	Two identical doors → "The destination never mattered. Only the fact that you were free to choose."
6. Creation	Constantly generating environments, evolving weapons, spawning enemies	Spirit of Creation (find the source, don't fight endlessly)	Aetherion	"If you were given the power to create a perfect world... would you erase the imperfect one?"
7. Destruction	Deteriorating weapons, collapsing platforms, resource management	Spirit of Destruction (protect what remains)	Nihros	"Tell me... what deserves to survive?"

[SEQ] Sequential (within each realm)

#	Task	Details
3.1	Level Design	Design realm layout in LDtk. Multiple interconnected rooms. Hidden paths, lore fragments, environmental storytelling.
3.2	Realm Mechanic	Implement the unique gameplay mechanic (e.g., time manipulation for Realm of Time). This affects physics, enemy behavior, puzzle logic.
3.3	Enemies & Hazards	Design 3-5 realm-specific enemy types. Each enemy reflects the realm's philosophy.
3.4	Realm Spirit Boss	Multi-phase boss fight with unique mechanics tied to the realm's law. Boss arena transforms during fight.
3.5	Primordial Dialogue	Post-boss philosophical conversation. No combat. Branching based on player's journey choices.

[PAR] Parallel (across realms)

#	Task	Details
---	------	---------

3.6	Realm Art Packages	Each realm has a unique tileset, color palette, background layers, particle effects. Can be created by art team while programmers work on previous realm's mechanics.
3.7	Realm Music	Each realm has: exploration theme, combat theme, boss theme, post-boss ambient. Composed in parallel.
3.8	Realm Lore Items	Collectible text/image fragments that reveal backstory. Written and integrated independently.

[WARN] IMPORTANT: **Realm overlap strategy:** While programmers implement Realm N's mechanics, artists can be creating Realm N+1's tilesets and animations, and the composer can be working on Realm N+1's music. This parallelism can save **8–12 weeks** total.

Critical Memory Scene (between Realms 5 & 6)

#	Task	Details
3.9	Cracks in Memory Cutscene	After leaving Realm of Chaos: Black Sword erupts, all wielders speak at once, then Kael sees his father standing beside Morvex, laughing like friends. Kael rejects the memory. This is a pivotal narrative turning point.

Phase 4 — The Core & Endings (Chapters 4–5) (Weeks 53–62)

Goal: Build the final act: the Seven gather, the Core revelation, the three endings, and epilogues.

[SEQ] Sequential Tasks

#	Task	Details	Deliverable
4.1	Seven Primordials Gathering Scene	All seven stand together for the first time. Chronar's speech: "You are the first to arrive knowing you do not [understand reality]." Gateway to the Core opens. Father's voice reveals identity.	Major cutscene
4.2	The Core Environment	Endless white light, floating memories. Unique art style: less pixel, more painterly/abstract. No enemies. Walk-and-experience environment.	The Core playable area

4.3	Memory Playback System	As Kael walks through the Core, he relives history: Golden Age, First Collapse, Seven's sacrifice, parents' choice. Interactive memories (player lives them, not just watches).	Memory system working
4.4	Truth Revelation Sequence	Core "speaks" through understanding. Parents' final memory: "If we are wrong... forgive us." Morvex reveals: "I killed them because they asked me to."	Emotional climax
4.5	Three Endings Implementation	Branch point after truth. Player chooses: Eternal Guardian / Age of Humanity / Broken Cycle. Each needs unique gameplay, cutscene, and epilogue.	3 complete endings
4.6	Epilogues	Short playable/cinematic epilogues for each ending. Final universal scene: child finds black flower, looks at horizon, smiles. Final narration text.	Game conclusion

[PAR] Parallel Tasks

#	Task	Details	Deliverable
4.7	Core Art & VFX	Unique visual style for the Core. White light shaders, memory fragment particles, painterly backgrounds.	Core visual assets
4.8	Core & Ending Music	Emotional orchestral pieces for: Core exploration, truth revelation, each ending, final narration.	5-6 music tracks
4.9	Choice Wielder Farewell Animations	Black Sword breaks apart → white light fragments → past wielders appear → father embraces Kael. Sprite animations + particle effects.	Farewell scene assets

Phase 5 — Polish, QA & Launch (Weeks 63–72)

Goal: Bug-fixing, balancing, optimization, marketing, and release.

[SEQ] Sequential Tasks

#	Task	Details
---	------	---------

5.1	Full Playthrough Testing	Play the entire game start-to-finish for each ending. Document bugs, pacing issues, difficulty spikes.
5.2	Balance Pass	Adjust combat difficulty, boss health/damage, puzzle timing. Implement adaptive AI (reads player habits, adjusts enemy behavior).
5.3	Performance Optimization	Profile with tracy . Optimize sprite batching, reduce draw calls, lazy-load realm assets. Target 60fps on minimum spec.
5.4	Accessibility Features	Colorblind modes, text size options, control remapping, subtitle customization.
5.5	Steam Page & Marketing	Store page, trailer video, screenshots, press kit. Free prologue demo (Nightmare + Village).
5.6	Release	Final build, Steam upload, launch day patch preparation.

[PAR] Parallel Tasks

#	Task	Details
5.7	Localization	Translate all dialogue/UI text. Separate text from code via .ron resource files.
5.8	Achievements & Speedrun Mode	Steam achievements, in-game completion tracker. "Trials of the Seven" boss rush mode.
5.9	Deluxe Edition Content	Digital soundtrack export, digital lore bible/artbook compilation.

How to Add Animations

Overview

All character and environment animations use **sprite sheet animation** — a single image containing all frames of an animation laid out in a grid.

Step-by-Step Pipeline

```
1. CREATE in Aseprite ■■■ Draw frames (e.g., 6 frames for idle, 8 for walk, 12 for attack) ■■■ Tag each animation (idle, walk, run, attack_light, attack_heavy, dodge, jump, fall, land, hurt, die) ■■■ Export as PNG sprite sheet (horizontal strip or grid)
2. DEFINE in Bevy ■■■ Create a TextureAtlas from the sprite sheet ■■■ Define AnimationClip structs: - name: "idle" - frames: [0, 1, 2, 3, 4, 5] - frame_duration: 0.12 seconds - loop_mode: Loop / Once / PingPong
3. CONTROL via Animation State Machine ■■■ States: Idle, Walking, Running, Jumping, Falling, Attacking, Dodging, Hurt, Dead ■■■ Transitions triggered by player input + physics state ■■■ Each state maps to an AnimationClip
4. RENDER in Bevy ■■■ SpriteSheetBundle with TextureAtlas handle ■■■ AnimationTimer component ticks every frame ■■■ On timer tick, advance frame index ■■■ Flip sprite horizontally based on facing direction
```

Animation Types in the Game

Animation Category	Examples	Frame Count (approx)	Notes
Kael (Player)	Idle, Walk, Run, Jump, Fall, Land, Light Attack, Heavy Attack, Dodge Roll, Parry, Hurt, Death	6–16 frames each	All directions (left/right via sprite flip)
Enemies	Patrol, Chase, Attack, Hurt, Death	4–12 frames each	Unique per enemy type
Realm Spirits (Bosses)	Idle, Phase transitions, Attack patterns (3–5 per boss), Defeat dissolve	12–24 frames each	Larger sprite sheets (64×64 or 128×128 px)
Primordials	Idle, Appear, Speak, Disappear	4–8 frames each	Emphasize presence over motion
Environment	Water flow, fire flicker, plant growth, time distortion shimmer, portal swirl	4–12 frames each	Tilemap animated tiles
VFX / Particles	Sword slash trail, dodge afterimage, hit spark, healing glow, Black Sword glow	4–8 frames each	Overlay sprites or shader-based

Realm-Specific Animations

Realm	Special Animations
Time	Objects aging/de-aging, ghostly echo replays of player's past actions, frozen raindrops
Space	Portals spiraling open, gravity-shift transitions, perspective-warp effects
Life	Plants growing in real-time, flowers blooming on footsteps, creatures evolving
Death	Souls fading in/out, memory fragments drifting, peaceful dissolution
Chaos	Glitch-frame animations, sprite corruption effects, random palette swaps
Creation	Mountains rising, bridges self-constructing, weapons morphing
Destruction	Crumbling platforms, weapon deterioration, scar-forming effects

Code Example (Bevy Rust)

```
// Define animation data #[derive(Component)] struct AnimationState { current: AnimationName, timer: Timer, frame_index: usize, } #[derive(Clone, Hash, Eq, PartialEq)] enum AnimationName { Idle, Walk, Run, Jump, Fall, AttackLight, AttackHeavy, Dodge, Hurt, Die, } // Animation clips loaded from asset files struct AnimationClip { frames: Vec<usize>, // indices into TextureAtlas frame_duration: f32, // seconds per frame looping: bool, } // System: advance animation frames fn animate_sprites( time: Res<Time>, mut query: Query<(&mut AnimationState, &mut TextureAtlas)>, clips: Res<AnimationClips>, ) { for (mut state, mut atlas) in &mut query { state.timer.tick(time.delta()); if state.timer.just_finished() { let clip = clips.get(&state.current); state.frame_index = (state.frame_index + 1) % clip.frames.len(); atlas.index = clip.frames[state.frame_index]; } }
```

How to Add Story Scenes

Overview

Story scenes are **in-engine dialogue encounters** — they happen within the game world (no pre-rendered video). Characters speak via text boxes while the game continues rendering the environment.

Story Scene Types

Type	When Used	Example
NPC Dialogue	Talking to villagers, past wielders' voices	Village elder telling Kael about his parents
Primordial Encounter	After each boss fight	Chronar asking: "Which moment made you believe revenge could heal grief?"
Black Sword Voices	Contextual moments during exploration	Past wielders debating Kael's choices
Memory Flashes	Triggered by story events	Kael's first clear memory of his parents
Environmental Storytelling	Inspecting objects/ruins	Reading ancient inscriptions in the Realms

Data Format (RON – Rusty Object Notation)

```
// assets/dialogues/chronar_meeting.ron DialogueSequence( id: "chronar_post_boss", trigger: AfterBoss("spirit_of_time"), scenes: [ DialogueLine( speaker: "Chronar", portrait: "chronar_exhausted", text: "Which moment made you believe revenge could heal grief?", emotion: Melancholy, duration: Auto, ), PlayerChoice( options: [ ("The night they died.", "chronar_response_grief"), ("I don't remember.", "chronar_response_honesty"), ("...", "chronar_response_silence"), ] ) DialogueLine( speaker: "Chronar", portrait: "chronar_fading", text: "Every answer has its proper time. Reach the end of your journey... then judge us.", emotion: Wisdom, duration: Auto, on_complete: FadeOut(2.0), ), ] )
```

Implementation Steps

```
1. WRITE the dialogue script in RON format 2. CREATE character portraits (multiple emotions per character) 3. LOAD via Bevy asset loader at scene start 4. RENDER dialogue box UI (portrait + name + text + choice buttons) 5. TRACK player choices in a ChoiceHistory resource 6. TRIGGER next scene or gameplay state based on outcome
```

How to Add Audio Tracks

Audio Categories

Category	Format	Channel	Examples
Music	OGG Vorbis, 44.1kHz stereo	Music channel	Village theme, Realm exploration, boss battle
Ambient	OGG Vorbis, looping	Ambient channel	Forest sounds, wind, water, Realm-specific ambience
SFX	WAV, 44.1kHz mono	SFX channel	Sword swing, footsteps, hit impact, UI click
Dialogue SFX	WAV, short bursts	Dialogue channel	Text blip sounds per character, Black Sword hum
Dynamic Boss Music	Multi-layer OGG	Music channel	Layers added/removed based on boss phase

Audio Pipeline

```
1. COMPOSE / RECORD ■■■ Use DAW (Reaper, LMMS, FL Studio) ■■■ Music: Orchestral samples (Spitfire LABS, BBC Symphony free tier) ■■■ SFX: Generate with sfar/jsfxr for pixel-style sounds ■■■ Export: Music → OGG Vorbis | SFX → WAV 2. ORGANIZE in assets/audio/ ■■■ assets/audio/music/village_day.ogg ■■■ assets/audio/music/realm_time_explore.ogg ■■■ assets/audio/music/boss_spirit_time.ogg ■■■ assets/audio/sfx/sword_swing_01.wav ■■■ assets/audio/ambient/forest_birds.ogg 3. INTEGRATE via bevy_kira_audio ■■■ AudioPlugin with channels: MusicChannel, SfxChannel, AmbientChannel ■■■ Music: play with crossfade on zone transitions ■■■ SFX: play on events (attack, jump, hit, UI) ■■■ Ambient: loop continuously, blend between zones 4. MIX dynamically ■■■ Boss fights: layer system (drums enter at Phase 2, choir at Phase 3) ■■■ Story moments: fade music to 30%, play dialogue SFX ■■■ Death: all audio fades except heartbeat
```

Music Track List

Track	Realm/Scene	Mood
Main Theme	Title screen	Epic, mysterious
Kael's Nightmare	Prologue	Distorted, fearful
Village of Memories	Village hub	Warm, nostalgic
Departure	Leaving village	Bittersweet, determined
Echoes of Time	Realm of Time	Ethereal, clockwork-like
Infinite Halls	Realm of Space	Wonder, vertigo
Garden of Forever	Realm of Life	Lush, overwhelming
White Silence	Realm of Death	Peaceful, mournful
Entropy's Dance	Realm of Chaos	Unpredictable, playful

The First Light	Realm of Creation	Hopeful, expansive
Dust and Flowers	Realm of Destruction	Somber, resilient
Heart of Reality	The Core	Sacred, transcendent
The Final Choice	Three endings	Emotionally open
Each boss	7 boss themes	Intense, realm-flavored

Realm-Specific Ambience

Realm	Ambient Sounds
Time	Ticking clocks, time distortion whooshes, reversed audio snippets
Space	Spatial echoes, gravitational hums, distant star-like tones
Life	Dense nature: insects, rustling leaves, water, birdsong (layered heavily)
Death	Near-silence, occasional distant wind, soft bell chimes
Chaos	Glitchy audio, random instrument bursts, overlapping sound fragments
Creation	Building sounds, crystalline growth tones, energetic hums
Destruction	Crumbling stone, distant thunder, wind through ruins

How to Add Story Clips (Cutscenes)

Overview

Cutscenes in The Last Primordial are **in-engine scripted sequences** — not pre-rendered videos. This keeps the visual style consistent and the file size small.

Cutscene Types

Type	Implementation	Example
Scripted Gameplay	Player control removed, camera moves, entities animate on script	Kael discovering the Black Sword
Illustrated Stills	Full-screen hand-drawn / pixel art images with text overlay and music	History of the Core during memory playback

Dialogue Cutscene	Characters face each other, close-up portraits, text boxes	Primordial meetings after boss fights
Memory Sequence	Screen effect (white flash, letterbox), replay of past events	Kael reliving parents' execution in the Core
Environmental Transition	Camera pans across a new realm, music swells, title card appears	Entering each new Realm

Cutscene Scripting System

```
// assets/cutscenes/nightmare_intro.ron CutsceneScript( id: "nightmare_prologue", steps: [ FadeIn(duration: 2.0, color: Black),
SetCamera(target: "burning_house", zoom: 1.5), PlayAudio(track: "nightmare_theme", channel: Music, fade_in: 1.0), wait(1.5),
SpawnEntity(id: "fire_particles", position: (120, 80)), ScreenShake(intensity: 3.0, duration: 2.0), ShowDialogue(speaker:
"???", text: "Kael...", portrait: "mother_silhouette"), wait(1.0), PanCamera(to: "seven_silhouettes", duration: 3.0, ease:
EaseInOut), ShowDialogue(speaker: "???", text: "Forgive us... one day, you'll understand.", portrait: "father_silhouette"),
ApplyShader(effect: ChromaticAberration, intensity: 0.8), FadeOut(duration: 1.5, color: White), TransitionTo(scene:
"village_bedroom"), ] )
```

Building a Cutscene Step-by-Step

```
1. STORYBOARD the cutscene ■■■ Sketch key frames on paper or in a tool ■■■ Define: camera positions, character movements,
dialogue, audio cues, VFX 2. CREATE ASSETS ■■■ Any unique sprites (silhouettes, illustrated stills) ■■■ Particle effects
(fire, light fragments, sword glow) ■■■ Audio cues (specific SFX for this moment) 3. SCRIPT in RON format ■■■ List sequential
steps: fade, camera, spawn, dialogue, wait, transition ■■■ Each step has a type and parameters 4. IMPLEMENT the CutsceneRunner
system in Bevy ■■■ Reads the script file ■■■ Executes steps sequentially ■■■ Disables player input during cutscene ■■■
Handles skip (fast-forward) if player presses a button 5. TEST timing ■■■ Adjust wait durations, camera speeds, fade lengths
■■■ Ensure dialogue is readable ■■■ Verify audio sync
```

Key Cutscenes to Build

#	Cutscene	Phase	Priority
1	Kael's Nightmare (opening)	Phase 2	[HIGH] Critical
2	Black Sword Discovery	Phase 2	[HIGH] Critical
3	Father's first whisper	Phase 2	[MED] High
4	Each Realm entrance (×7)	Phase 3	[MED] High
5	Each Realm Spirit defeat (×7)	Phase 3	[MED] High
6	Each Primordial meeting (×7)	Phase 3	[HIGH] Critical
7	Cracks in Memory (father + Morvex vision)	Phase 3	[HIGH] Critical
8	Seven Primordials Gathering	Phase 4	[HIGH] Critical
9	Father reveals identity	Phase 4	[HIGH] Critical

10	Core memory playback (Golden Age, Collapse, Sacrifice)	Phase 4	[HIGH] Critical
11	Parents' true execution memory	Phase 4	[HIGH] Critical
12	Black Sword dissolution + wielder farewell	Phase 4	[HIGH] Critical
13	Three ending sequences	Phase 4	[HIGH] Critical
14	Epilogues (x3) + black flower final scene	Phase 4	[MED] High

How to Add Characters

Character Pipeline

```
1. DESIGN the character ■■■ Concept art (hand-drawn sketches) ■■■ Define: size (e.g., 32x48 px), color palette, visual motifs ■■■ Personality traits → visual language (exhausted Chronar, scarred Nihros, shifting Xael) 2. CREATE sprite sheet in Aseprite ■■■ All animation states on one sheet ■■■ Standard frame sizes (consistent across all characters) ■■■ Export: PNG sprite sheet + JSON metadata 3. DEFINE in code ■■■ Character component with stats (health, speed, damage) ■■■ Animation state machine component ■■■ AI component (for enemies/NPCs) ■■■ Dialogue data reference (for speaking characters) 4. CREATE portraits ■■■ Larger detailed artwork (e.g., 128x128 px) for dialogue boxes ■■■ Multiple emotion variants: neutral, angry, sad, happy, mysterious, exhausted ■■■ Each Primordial needs 3-5 portrait variants 5. REGISTER in entity spawner ■■■ Function to spawn character at position with all components ■■■ Add to level data (LDTK entity layer) or cutscene script 6. IMPLEMENT behavior ■■■ NPCs: dialogue triggers, patrol paths, schedule (day/night) ■■■ Enemies: AI state machine, attack patterns, realm-specific behavior ■■■ Bosses: multi-phase AI, arena interactions, defeat sequence ■■■ Primordials: appear/disappear effects, no combat AI, dialogue-only
```

Character Roster

Character	Type	Sprite Size	Animations Needed	Portraits
Kael	Player	32x48	15+ states	5 emotions
Village NPCs (x8-10)	NPC	32x48	3 states (idle, walk, talk)	2 emotions each
Village Elders (x2-3)	NPC	32x48	3 states	3 emotions each
Realm Enemies (x3-5 per realm)	Enemy	32x32 to 48x48	5 states (patrol, chase, attack, hurt, die)	None
Realm Spirits (x7)	Boss	64x64 to 128x128	8-12 states (idle, attacks, phase shifts, defeat)	2 emotions each
Chronar	Primordial	48x64	4 states (appear, idle, speak, fade)	4 emotions
Vastara	Primordial	48x64	4 states	3 emotions

Elaria	Primordial	48×64	4 states	4 emotions
Morvex	Primordial	48×64	4 states	5 emotions (most emotional arc)
Xael	Primordial	48×64 (shifting)	6 states (appearance changes)	3 emotions
Aetherion	Primordial	48×64	4 states	3 emotions
Nihros	Primordial	48×64	4 states	3 emotions
Past Choice Wielders (×5-8 shown)	Sword Voice	None (portraits only)	None	2 emotions each
Kael's Father	Sword Voice / Memory	32×48 (memory scenes)	3 states	3 emotions
Kael's Mother	Memory	32×48 (memory scenes)	2 states	2 emotions

Primordial Visual Design Guidelines (from Lore Bible)

Primordial	Visual Direction
Chronar (Time)	Appears exhausted despite immortality. Eyes that have seen too much. Subtle clock-like motifs.
Vastara (Space)	Ethereal, slightly translucent. Stars visible within her form. Soft-spoken presence.
Elaria (Life)	Surrounded by small blooming flowers. Compassionate but overwhelmed expression. Warm green/gold palette.
Morvex (Death)	Emotionally distant, not evil. Gentle eyes filled with ancient guilt. Muted, pale palette.
Nihros (Destruction)	Covered in scars from every necessary destruction. Powerful build, weary posture. Dark reds/grays.
Aetherion (Creation)	Surrounded by tiny stars and light. Warm, hopeful expression. Gold/white palette.
Xael (Chaos)	Appearance subtly shifts — never exactly the same twice. Playful expression. Multicolor glitch effects.

Dependency & Parallelism Map

Master Dependency Graph

[GRAPH] DEPENDENCY GRAPH (Mermaid Diagram)

Render this in a Mermaid-compatible viewer for the interactive graph.

P0_1 (0.1 Rust/Bevy Setup) → P0_2 (0.2 Version Control)
P0_2 → P0_3 (0.3 Folder Structure)
P0_3 → P0_4 (0.4 GDD Finalization)
P0_4 → P0_5 (0.5 Art Direction)
P0_4 → P0_6 (0.6 Audio Direction)
P0_4 → P0_7 (0.7 Narrative Script)
P0_4 → P0_8 (0.8 Tool Selection)
P0_3 → P1_1 (1.1 Player Movement)
P1_1 → P1_2 (1.2 Camera System)
P1_2 → P1_3 (1.3 Combat System)
P1_3 → P1_4 (1.4 Enemy AI)
P1_2 → P1_5 (1.5 Tilemap Loading)
P0_3 → P1_6 (1.6 UI System)
P0_7 → P1_7 (1.7 Dialogue System)
P0_3 → P1_8 (1.8 Save/Load)
P0_5 → P1_9 (1.9 Sprite Pipeline)
P0_6 → P1_10 (1.10 Audio Engine)
P1_5 → P2_1 (2.1 Nightmare Sequence)
P1_7 → P2_1
P1_10 → P2_1
P2_1 → P2_2 (2.2 Village Hub)
P2_2 → P2_3 (2.3 NPC Interaction)
P2_3 → P2_4 (2.4 Black Sword Event)
P2_4 → P2_5 (2.5 Departure)
P1_7 → P2_8 (2.8 Sword Voice System)
P0_5 → P2_6 (2.6 Village Art)
P0_6 → P2_7 (2.7 Village Music)
P1_10 → P2_9 (2.9 Nightmare VFX)
P2_5 → P3 (Phase 3: Seven Realms)
P3 → P4 (Phase 4: Core & Endings)
P4 → P5 (Phase 5: Polish & Launch)

Parallelism Summary Table

Phase	Parallel Tracks	Max Parallel Tasks
Phase 0	Art direction ■ Audio direction ■ Narrative script ■ Tool selection	4
Phase 1	(UI ■ Dialogue ■ Save/Load ■ Sprite Pipeline ■ Audio) alongside sequential core engine	5
Phase 2	(Village Art ■ Village Music ■ VFX) alongside sequential gameplay	3
Phase 3	Realm N programming ■ Realm N+1 art ■ Realm N+1 music	3

Phase 4	Core art ■ Core music ■ Farewell animations alongside sequential story	3
Phase 5	Localization ■ Achievements ■ Deluxe content alongside sequential QA	3

Color Legend for Task Dependencies

Color	Meaning
[HIGH] Red	Blocking — nothing else can proceed until this is done
[MED] Yellow	Important but has parallel alternatives
[LOW] Green	Fully parallelizable with other tasks

Tool & Asset Pipeline Summary

Purpose	Tool	Format
Game Engine	Bevy (Rust)	—
Pixel Art & Animation	Aseprite	PNG sprite sheets + JSON metadata
Level Design	LDtk (Level Designer Toolkit)	.ldtk → parsed via bevy_ecs_ldtk
Audio Production	Reaper / LMMS / FL Studio	OGG (music) / WAV (SFX)
SFX Generation	jsfxr / sfxr	WAV
Dialogue Data	Custom RON files	.ron
Cutscene Scripts	Custom RON files	.ron
Version Control	Git + GitHub	—
Physics	bevy_rapier2d	—
Audio Playback	bevy_kira_audio	—
UI	bevy_ui or bevy_egui	—
Profiling	Tracy	—
CI/CD	GitHub Actions	cargo check, clippy, test

[INFO] NOTE: ****Estimated Total Development Time:**** ~72 weeks (~18 months) for a 3-person team working full-time. Adjust timelines based on team availability and scope decisions. Consider releasing the ****free prologue demo**** (Nightmare + Village) at Week 30 to build community hype while continuing realm development.

[TIP] TIP: ****Recommended first milestone:**** Complete Phase 1 (Vertical Slice) and have playable combat + dialogue + one test area by Week 18. This validates your tech stack before committing to full content production.